

Automated Compositional Verification of Hyperproperties

Project Description for a Bachelor's Thesis

Patrick Neugebauer

April 4, 2025

Supervisors: Thibault Dardinier, Anqi Li

Group: Programming Methodology Group, ETH Zurich

1 Introduction

Many properties of interest are hyperproperties [1], these are properties that relate multiple executions of a program. For example, properties such as monotonicity or determinism are hyperproperties.

To prove that a program satisfies certain hyperproperties, one can use a proof system such as Hyper Hoare Logic (HHL) [3]. HHL establishes hyper-triples of the form $\{P\} C \{Q\}$, where P (the precondition) and Q (the postcondition) are predicates over **sets of states**, unlike standard Hoare Logic [5][4], where P and Q are predicates over **states**.

With HHL, we can reason about determinism with the following hyper-triple: $\{\forall \langle \sigma_1 \rangle, \langle \sigma_2 \rangle. \sigma_1(x) = \sigma_2(x)\} C \{\forall \langle \sigma_1 \rangle, \langle \sigma_2 \rangle. \sigma_1(y) = \sigma_2(y)\}$ in this example C could be a deterministic function with the input x and the output y , for example $y := x + 1$.

To automatically verify hyperproperties, a deductive program verifier called Hypra [2] that is based on HHL has been developed. It translates the Hypra code to Viper [6] code by tracking sets of states over the program.

2 Limitations of Hypra

Since Hypra is a relatively new program verifier, it does not yet support every aspect of HHL. We can see some of the functionality needed in the following example:

```
method det(x : Int)
returns (y : Int)
  requires  $\forall \langle \sigma_1 \rangle, \langle \sigma_2 \rangle. \sigma_1(x) = \sigma_2(x) :: p_1$ 
  ensures  $\forall \langle \sigma_1 \rangle, \langle \sigma_2 \rangle. \sigma_1(y) = \sigma_2(y) :: p_1$ 
{
  y := x + 1
}

method main()
returns (u : Int, v : Int)
  ensures  $\forall \langle \sigma \rangle. \sigma(u) = \sigma(v)$ 
{
  u := det(3) @a
  v := det(3) @b
  apply p1 to a ∪ b
}
```

Figure 1: In this example we want to show that the a deterministic method (det) applied twice to the same input results in the same output. For this to work we need novel syntax (in blue) and new functionality for Hypra.

In Figure 1 we want to verify that the output of a deterministic function (det) on the same input twice results in the same output. Although this appears logically sound, the verification process fails. Currently, for each function call Hypra checks if the precondition holds and then assumes the postcondition individually; however, Hypra never verifies the determinism property on the two method calls at the same time and therefore cannot reason about how the return values correlate to each other. Therefore, the postcondition of main cannot be verified.

3 Our Approach

Our main goal is to track and correlate various method calls that interact with a specific method specifications. Figure 1 shows an example of what we would be able to do with these features. To enable this, we will introduce a Domain-Specific Language (DSL) that provides a mechanism for referencing which specification should be applied to which method call. This DSL allows us to label specific method calls and name certain method properties so that we can apply the right specification on the labeled method calls at a specific point in the program. With the DSL, we also want to introduce unions, filters and the possible use of logical variables on our method property application.

Once we have this DSL, we would be able to reason about compositionality of method specifications with Hypra; for example, we would be able to verify Figure 1. To do so, Hypra would capture all the sets of states before and after each function call. Then once we tell it to apply a specification (e.g. `apply  $p_1$  to  $a \cup b$`) it would assert the precondition of the function’s specification (e.g. p_1) on the set of states before each of the two function calls. Then, it would assume the postcondition of the specification (e.g. p_1) on the set of states after both calls.

We will go into further detail why we want each of those capabilities later in the next section.

4 Goals

The goals of this project are as follows.

4.1 Core Goals

Here are the main goals of the thesis, which will be achieved as part of this research.

4.1.1 Logical Variables, Named Specifications and Abstract Methods

- **Support for Logical Variables:**

Support for logical variables is needed in order to apply certain method specifications correctly on method calls; for example it would be useful for applying a monotonicity property on two calls of a monotonic function (see the fibonacci example in the HHL paper [3] figure 7).

- **Named Specifications:**

Assigning names to specifications provides the ability to reference them later in the DSL when we apply them on method calls.

- **Abstract Methods:**

We also want to have support for abstract methods.

4.1.2 A Domain-Specific Language for Method Specifications and Labels for Function Calls

We require a DSL that supports performing unions on our states, useful to link multiple function calls to each other (see Figure 1). This DSL should also be able to rename logical variables so we can effectively use them in our method specification, and apply projections on our states so that certain properties that only hold for a subset of states can be verified. To achieve this, we need labels to reference our method calls.

4.1.3 Evaluation

We will evaluate the novel features of the new version of Hypra by developing examples to grasp its capabilities, correctness, and limitations.

4.2 Extension Goals

These are goals that further enhance Hypra’s capabilities.

4.2.1 Compositional Verification Across Loop Iterations

It is not trivial how method specifications would extend to loops. For example, how would method calls from different loop iterations be referenced later on? It would be necessary to investigate the appropriate syntax and implementation strategies for this functionality.

4.2.2 Higher-Order Function

We would like to extend Hypra to support higher-order functions, for example this would be useful if we want to verify that a map function applied to a monotonic function on a sorted list would result in another sorted list.

4.2.3 Stronger Renaming Capabilities of the DSL

Stronger renaming capabilities in the DSL would also make Hypra more expressive, with that we could prove that the sequential composition of a function with a minimum and a monotonic function still has a minimum (example from figure 12 in the HHL paper [3]).

4.2.4 More Improvements to Hypra

Exploring other general improvements, that would make Hypra more expressive.

References

- [1] Michael R. Clarkson and Fred B. Schneider. “Hyperproperties”. In: *2008 21st IEEE Computer Security Foundations Symposium*. 2008, pp. 51–65. DOI: 10.1109/CSF.2008.7.
- [2] Thibault Dardinier, Anqi Li, and Peter Müller. “Hypra: A Deductive Program Verifier for Hyper Hoare Logic”. In: *Proceedings of the ACM on Programming Languages* 8 (Oct. 2024), pp. 1279–1308. DOI: 10.1145/3689756. URL: https://dardinier.me/papers/00PSLA24_Hypra.pdf.
- [3] Thibault Dardinier and Peter Müller. “Hyper Hoare Logic: (Dis-)Proving Program Hyperproperties”. In: *Proceedings of the ACM on Programming Languages* 8.PLDI (June 2024), pp. 1485–1509. ISSN: 2475-1421. DOI: 10.1145/3656437. URL: <http://dx.doi.org/10.1145/3656437>.
- [4] Robert W. Floyd. “Assigning Meanings to Programs”. In: *Proceedings of Symposium in Applied Mathematics (1967)*, 19-32. 1967.
- [5] C. A. R. Hoare. “An axiomatic basis for computer programming”. In: *Commun. ACM* 12.10 (Oct. 1969), pp. 576–580. ISSN: 0001-0782. DOI: 10.1145/363235.363259. URL: <https://doi.org/10.1145/363235.363259>.
- [6] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. “Viper: A Verification Infrastructure for Permission-Based Reasoning”. In: *Verification, Model Checking, and Abstract Interpretation*. Ed. by Barbara Jobstmann and K. Rustan M. Leino. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 41–62. ISBN: 978-3-662-49122-5.